Cairo University
Faculty of Computer and Artificial Intelligence

CS453: Software Testing and Quality Assurance
2025-2026

Assignment 1
Part 2: Privacy-First AI Chat Infrastructure

Course Instructor
Dr. Manar ElKady

Team Members:

Name	ID
Ebaa Mostafa	20220001
Touka Mohamed	20220094
Yasmine Sherif	20220377
Muhammad Hassan	20221120

Assignment 1 1
Part 2: Privacy-First AI Chat Infrastructure 1
1. Overview 3
2. Test Summary 3
3. Test Case Analysis 4
4. Behavioral Verification 7
MainScrubber-1: Full Scrubbing Workflow 7
MainScrubber-2: Component Isolation — Digit-Only Mode 7
MainScrubber-3: Component Isolation — Email-Only Mode 8
5. Severity Assessment 8
6. Conclusion 9

1. Overview

This report summarizes the testing results for:

- DigitScrubber
- EmailScrubber
- MainScrubber: orchestrator for both classes

Testing approach:

- **Functional testing**
  Verifying correctness of outputs for various input scenarios.

- **Behavioral verification**
  Ensuring correct interaction between components
  - Method invocation
  - Invocation count
  - Data flow between dependencies

2. Test Summary

Component	Total Tests	Passed	Failed	Status
DigitScrubber	18	14	4	Defective
EmailScrubber	14	10	4	Defective
MainScrubber	10	6	4	Defective
Total	42	30	12	Issues detected

3. Test Case Analysis

Test case ID	Test class	Test case	Expected	Actual	Status
TC-01	DigitScrubber	null	NullPointerException	NullPointerException	Passed
TC-02	DigitScrubber	""	IllegalArgumentException	IllegalArgumentException	Passed
TC-03	DigitScrubber	" "	IllegalArgumentException	IllegalArgumentException	Passed
TC-04	DigitScrubber	"\n\t"	IllegalArgumentException	IllegalArgumentException	Passed
TC-05	DigitScrubber	"1234"	"XXXX"	"XXXX"	Passed
TC-06	DigitScrubber	"ID 123"	"ID XXX"	"ID XXX"	Passed
TC-07	DigitScrubber	"Hello World"	"Hello World"	"Hello World"	Passed
TC-08	DigitScrubber	"Cost is 50$"	"Cost is 50$"	"Cost is XX$"	Failed
TC-09	DigitScrubber	"Cost is 50$ and 123"	"Cost is 50$ and XXX"	"Cost is XX$ and XXX"	Failed
TC-10	DigitScrubber	"Cost is 50 $"	"Cost is XX $"	"Cost is XX $"	Passed
TC-11	DigitScrubber	"abc50$xyz"	"abc50$xyz"	"abcXX$xyz"	Failed
TC-12	DigitScrubber	"1"	"X"	"X"	Passed
TC-13	DigitScrubber	"1-2-3"	"X-X-X"	"X-X-X"	Passed
TC-14	DigitScrubber	"1abc2"	"XabcX"	"XabcX"	Passed
TC-15	DigitScrubber	"12 34 56"	"XX XX XX"	"XX XX XX"	Passed

TC-16	DigitScrubber	"1\n2\n3"	"X\nX\nX"	"X\nX\nX"	Passed
TC-17	DigitScrubber	"12345678901234567890"	"XXXXXXXXXXXXXXXXXXXXX"	"XXXXXXXXXXXXXXXXXXXXX"	Passed
TC-18	DigitScrubber	"User123 has 4 items costing 50$ each."	"UserXXX has X items costing 50$ each."	"UserXXX has X items costing XX$ each."	Failed
TC-19	EmailScrubber	null	NullPointerException	NullPointerException	Passed
TC-20	EmailScrubber	""	IllegalArgumentException	NullPointerException	Failed
TC-21	EmailScrubber	" "	IllegalArgumentException	NullPointerException	Failed
TC-22	EmailScrubber	"\n\t"	IllegalArgumentException	NullPointerException	Failed
TC-23	EmailScrubber	"Contact me at test@example.com"	"Contact me at [EMAIL_HIDDEN]"	"Contact me at [EMAIL_HIDDEN]"	Passed
TC-24	EmailScrubber	"a@test.com b@test.com"	"[EMAIL_HIDDEN] [EMAIL_HIDDEN]"	"[EMAIL_HIDDEN] [EMAIL_HIDDEN]"	Passed
TC-25	EmailScrubber	"My email is test@example.com for more info."	"My email is [EMAIL_HIDDEN] for more info."	"My email is [EMAIL_HIDDEN] for more info."	Passed
TC-26	EmailScrubber	"Hello world"	"Hello world"	"Hello world"	Passed
TC-27	EmailScrubber	"test@com"	"test@com"	"test@com"	Passed
TC-28	EmailScrubber	"user@mail.example.com"	"[EMAIL_HIDDEN]"	"[EMAIL_HIDDEN]"	Passed
TC-29	EmailScrubber	"user+tag@example.com"	"[EMAIL_HIDDEN]"	"[EMAIL_HIDDEN]"	Passed
TC-30	EmailScrubber	"user.name@example.com"	"[EMAIL_HIDDEN]"	"[EMAIL_HIDDEN]"	Passed

TC-31	EmailScrubber	"user123@example.com"	"[EMAIL_HIDDEN]"	"[EMAIL_HIDDEN]"	Failed
TC-32	EmailScrubber	"TEST@EXAMPLE.COM"	"[EMAIL_HIDDEN]"	"[EMAIL_HIDDEN]"	Passed
TC-33	MainScrubber	null, FULL_SCRUBBING	NullPointerException	null	Failed
TC-34	MainScrubber	"", FULL_SCRUBBING	IllegalArgumentException	null	Failed
TC-35	MainScrubber	" ", FULL_SCRUBBING	IllegalArgumentException	null	Failed
TC-36	MainScrubber	"\n\t", FULL_SCRUBBING	IllegalArgumentException	null	Failed
TC-37	MainScrubber	"user@test.com 123", FULL_SCRUBBING	"[EMAIL_HIDDEN] XXX", both called once	"[EMAIL_HIDDEN] XXX", both called once	Passed
TC-38	MainScrubber	"user@test.com 123 another@test.com 456", FULL_SCRUBBING	"[EMAIL_HIDDEN] XXX [EMAIL_HIDDEN] XXX"	"[EMAIL_HIDDEN] XXX [EMAIL_HIDDEN] XXX"	Passed
TC-39	MainScrubber	"123", ONLY_DIGITS	"XXX", email scrubber not called	"XXX", email scrubber not called	Passed
TC-40	MainScrubber	"user@test.com", ONLY_EMAILS	"[EMAIL_HIDDEN]", digit scrubber not called	"[EMAIL_HIDDEN]", digit scrubber not called	Passed
TC-41	MainScrubber	"hello", FULL_SCRUBBING	"hello", both called	"hello", both called	Passed
TC-42	MainScrubber	"user123@test.com", FULL_SCRUBBING	"[EMAIL_HIDDEN]"	"[EMAIL_HIDDEN]"	Passed

4. Behavioral Verification

MainScrubber-1: Full Scrubbing Workflow

Test Method: testScrub_FullScrubbing_CallsBothDependencies

Objective: Verify that when FULL_SCRUBBING mode is active, the Main Scrubber correctly orchestrates the data flow sequentially through both scrubbing dependencies.

Execution Steps:

Input a mixed string containing digits and an email (user@test.com 123).

Invoke Main Scrubber with ScrubMode.FULL_SCRUBBING.

Expected Behavior: The system successfully passes the raw input to the Digit Scrubber.

Chaining: The system takes the intermediate output from the Digit Scrubber and passes it to the Email Scrubber for final masking.

Execution Volume (Efficiency): The system invokes both the Digit Scrubber and the Email Scrubber exactly once. Zero redundant calls or processing loops occur.
Actual Result: System successfully chained both services and returned the fully masked string.

Status: Passed

MainScrubber-2: Component Isolation — Digit-Only Mode

Test Method: testScrub_OnlyDigits_DoesNotCallEmailScrubber

Objective: Verify that when ONLY_DIGITS mode is selected, the system restricts processing entirely to the Digit service and guarantees the Email component is bypassed.

Execution Steps:

Input a string containing digits (123).

Invoke Main Scrubber with ScrubMode.ONLY_DIGITS.

Expected Behavior: Primary Action: The system successfully routes the input to the Digit Scrubber.

Isolation Guarantee: The orchestrator enforces strict component isolation, guaranteeing zero invocations of the out-of-scope Email service.

Actual Result: Digit scrubber processed the string; zero interactions occurred with the Email scrubber.

Status: Passed

MainScrubber-3: Component Isolation — Email-Only Mode

Test Method: testScrub_OnlyEmails_DoesNotCallDigitScrubber

Objective: Verify that when ONLY_EMAILS mode is selected, the system restricts processing entirely to the Email service and guarantees the Digit component is bypassed.

Execution Steps:

Input a string containing an email (user@test.com).

Invoke Main Scrubber with ScrubMode.ONLY_EMAILS.

Expected Behavior: Primary Action: The system successfully routes the input to the Email Scrubber.

Isolation Guarantee: The orchestrator enforces strict component isolation, guaranteeing zero invocations of the out-of-scope Digit service.

Actual Result: Email scrubber processed the string; zero interactions occurred with the Digit scrubber.

Status: Passed

5. Severity Assessment

Defect	Severity	Reason
DigitScrubber: price patterns are not kept unmasked, all digits get replaced including those before '$'	High	Core functional requirement explicitly stated in the requirements. Directly causes data to be incorrectly masked in production.
EmailScrubber: blank/empty input throws NullPointerException instead of IllegalArgumentException	High	Violates the interface contract defined in the spec.
MainScrubber: exceptions are not thrown and methods return null instead.	Medium	Affects observability, as the caller cannot differentiate between null input, blank input, or any internal failure without additional logging or error messages which complicates debugging.

6. Conclusion

Testing was conducted across three components (DigitScrubber, EmailScrubber, MainScrubber) covering 42 test cases combining functional and behavioural verification, 30 tests passed and 12 failed, with defects identified in all three components.

DigitScrubber is the most functionally defective component. Its core regex masks all digits unconditionally, violating the explicit price protection requirement. All other behavior including exception handling, mixed text, and edge case formatting is correct.

EmailScrubber has a contract violation in its input validation layer. Blank and empty inputs throw NullPointerException instead of the specified IllegalArgumentException, meaning callers relying on the interface contract will not catch these failures correctly. Email pattern matching itself is functioning correctly across all tested formats.

MainScrubber passes all behavioral verification tests: mode routing, dependency chaining, and component isolation all behave as specified. The only defect is that it silently returns null for invalid inputs instead of throwing the found exception, reducing the caller's ability to distinguish failure causes.